Model monitoring and logging

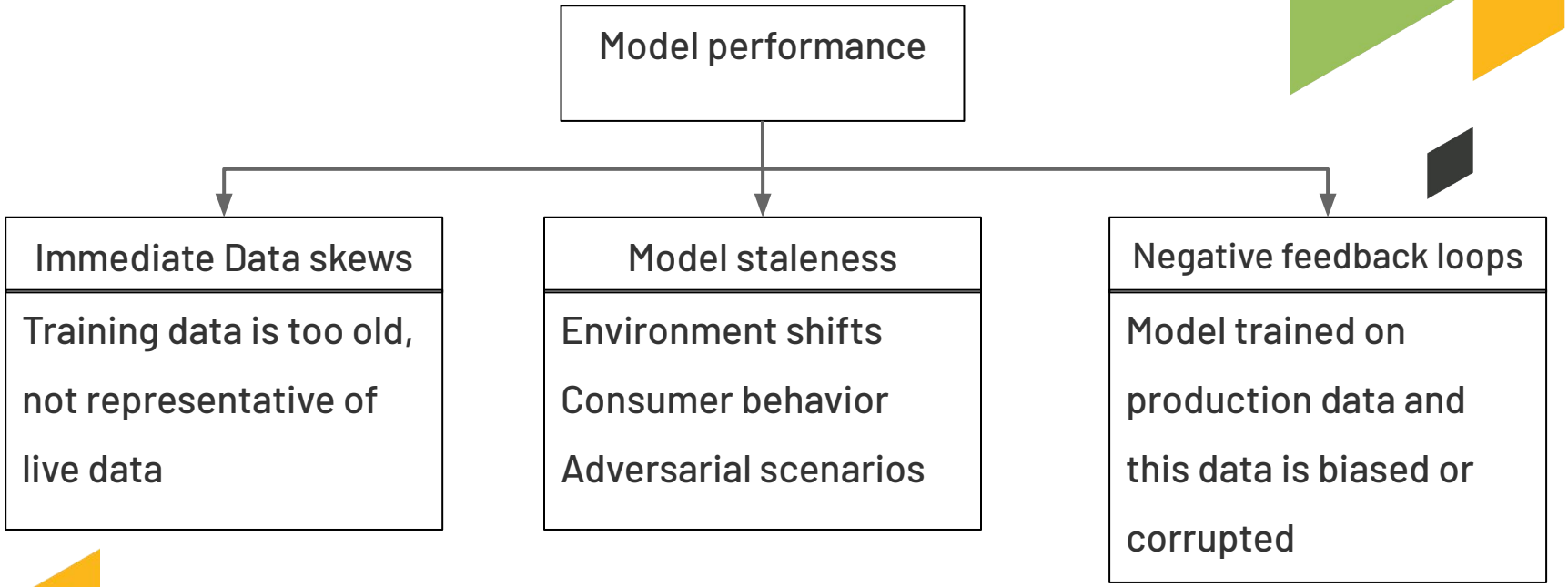
by LAB Team

ML Lifecycle

System and infrastructure

Model performance

Reasons to monitor model performance



ML vs. System monitoring

ML Monitoring (functional monitoring)	System monitoring (non-functional monitoring)
Predictive performance	System performance
Changes in serving data	System status
Metrics used during training	System reliability
Characteristics of features	

Observability

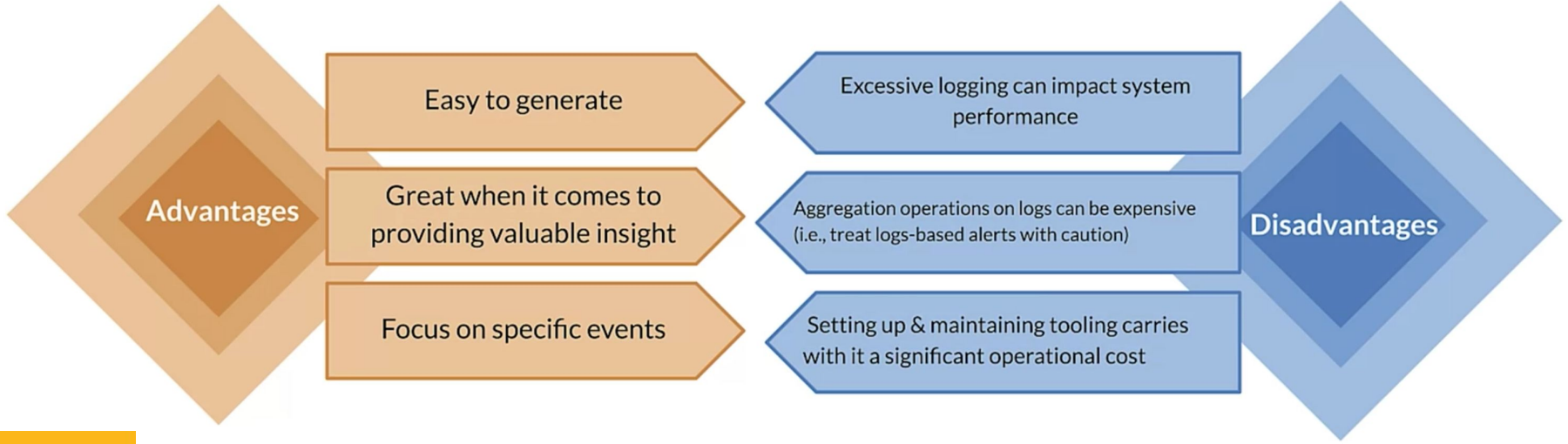
- Observability measure how well the internal states of a system can be inferred by knowing the inputs and outputs.
- Observability is about making measurements.
  - Not only the top-level metrics: slice data into multiple subsets to measure
  - Domain knowledge is important for observability: knowledge to understand how to slice data into subsets
  - Supervised and unsupervised analysis:
    - supervised settings: the predictions vs. the labels
    - unsupervised settings: the mean, the median, the range ... of the input features

Observability

- The goal of observability:
  - Alertable:
    - Metrics and threshold designed to raise alert
  - Actionable:
    - Correct the failure of the model and system

Logging

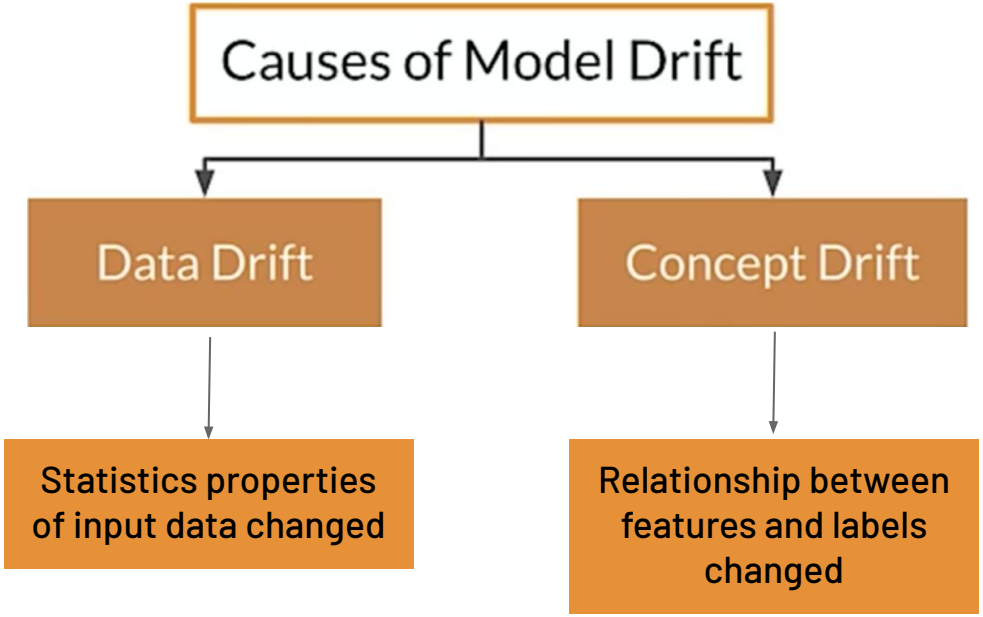
- An event log is an immutable, time-stamped record of discrete events that happened over time.



Storing log data

- Parsing and storing log data in a queryable format enables analysis
  - Extracting values to generate distributions and statistics
  - Ordering by timestamps to recognize the trend or seasonality
  - Identifying the system failure
- Enabling automated reporting, dashboards and alerting.

Model decay


Causes of Model Drift
Data Drift
Concept Drift
Statistics properties of input data changed
Relationship between features and labels changed

Detecting model decay

- Log predictions
  - Log incoming requests
    - Can be used to check data drift
  - Log both incoming requests and prediction responses
    - Can be used to check concept drift
  - Log the groundtruth (if possible)
    - Can be used as new training data

Detecting model decay

- Detecting the drift
  - Analyze the statistical properties of logged data (incoming request, prediction response and groundtruth)
  - Deploy the statistical properties to dashboard for observation

Mitigating model decay

- Notify operational and business stakeholders about the decay
- Take steps to bring model back to acceptable performance
  - Prepare a better dataset
  - Train a model

Mitigating model decay

- Prepare a better dataset
  - Determine the portion of training data is still correct (some method such as KL divergence, JS divergence or KS test)
  - Keep the good data, discard the bad and add new data -OR-
  - Discard the data before a certain date and add new data -OR-
  - Create an entire new data

Note: choose the method from the above based on the ability to collect the new labeled data

Mitigating model decay

- Train a model
  - Finetune a model from the checkpoint using new dataset -OR-
  - Re-initialize model and train from scratch

Note: choose the method from the above based on the amount of data and how far has data drifted (or try both and compare)

Re-training policy

On-Demand	• Manually re-train the model
On a Schedule	• New labelled data is available at a daily, weekly or monthly basis
Availability of New Training Data	• New data available on ad-hoc basis, when it is collected and available in source database

-> We should automate this process!!!

Re-thinking while re-training

- Should we re-design the entire training pipeline? (both data and model)
- Should we use feature engineering or feature selection? (to improve both accuracy and performance)
- Should we train model from scratch or finetune? (to learn a lot from new data or keep the old effect)
- Should we re-design the model architecture? (to try the new technology)

-> This can be a chance to improve our model significantly!!!


PIXTA
VIETNAM